ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук

Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Приглашённый преподаватель ФКН НИУ ВШЭ

_____ Веселко Никита Игоревич

«___» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель образовательной программы "Программная инженерия", старший преподаватель департамента программной инженерии

_____ Н. А. Павлочев

«___» _____ 2026 г.

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

КРОСС-ПРОДУКТОВАЯ ОН-ЧЕЙН СИСТЕМА ЛОЯЛЬНОСТИ «LOYALX».

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.09.09-01 51 01-1-ЛУ

Исполнители:

Студент группы БПИ246

_____ / А. А. Дулаев /

«___» _____ 2026 г.

2026

УТВЕРЖДЕН

RU.17701729.09.09-01 51 01-1-ЛУ

КРОСС-ПРОДУКТОВАЯ ОН-ЧЕЙН СИСТЕМА ЛОЯЛЬНОСТИ «LOYALX».

Программа и методика испытаний

RU.17701729.09.09-01 51 01-1

Листов 15

2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

АННОТАЦИЯ

Программа и методика испытаний — программный документ, регламентирующий требования, подлежащие проверке при испытаниях программы, а также порядок и методы их контроля.

Настоящая Программа и методика испытаний программы «Кросс-продуктовая он-чейн система лояльности «LoyalX»» разработана в соответствии с требованиями ГОСТ 19.301-79 [5] и содержит следующие разделы: «Объект испытаний», «Цель испытаний», «Требования к программе», «Требования к программной документации», «Средства и порядок испытаний», «Методы испытаний».

В разделе «Объект испытаний» указано наименование, область применения и обозначение испытываемой программы.

В разделе «Цель испытаний» указана цель проведения испытаний.

Раздел «Требования к программе» содержит технические требования, проверка которых должна быть осуществлена в ходе испытаний.

Раздел «Требования к программной документации» содержит состав предъявляемой на испытания программной документации.

Раздел «Средства и порядок испытаний» содержит сведения о технических и программных средствах, используемых при проведении испытаний, а также порядок и последовательность проведения испытаний.

Раздел «Методы испытаний» содержит описание используемых методов испытаний и тестовые примеры с описанием исходных данных, контрольных результатов и условий выполнения.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 [1]: Виды программ и программных документов.
2. ГОСТ 19.103-77 [2]: Обозначения программ и программных документов.
3. ГОСТ 19.105-78 [3]: Общие требования к программным документам.
4. ГОСТ 19.106-78 [4]: Требования к программным документам, выполненным печатным способом.
5. ГОСТ 19.301-79 [5]: Программа и методика испытаний. Требования к содержанию и оформлению.

Изменения к настоящей Программе и методике испытаний оформляются согласно ГОСТ 19.603-78 [6], ГОСТ 19.604-78 [7].

СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ 4
2. ЦЕЛЬ ИСПЫТАНИЙ 5
3. ТРЕБОВАНИЯ К ПРОГРАММЕ 6
3.1. Требования, проверяемые в ходе испытаний 6
3.2. Требования к среде испытаний 8
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ 9
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ 10
5.1. Технические средства 10
5.2. Программные средства 10
5.3. Порядок проведения испытаний 10
6. МЕТОДЫ ИСПЫТАНИЙ 11
6.1. Тестовый пример Т1 — Подключение кошелька (Ф1) 11
6.2. Тестовый пример Т2 — Регистрация бренда (Ф2) 11
6.3. Тестовый пример Т3 — Эмиссия баллов (Ф3) 11
6.4. Тестовый пример Т4 — Начисление по QR-коду (Ф4, Н1) 11
6.5. Тестовый пример Т5 — Списание по QR-коду (Ф5) 12
6.6. Тестовый пример Т6 — Согласование курса между разными владельцами (Ф7) 12
6.7. Тестовый пример Т7 — Согласование курса между своими брендами (Ф8) 12
6.8. Тестовый пример Т8 — Обмен баллов (Ф6, Н2) 12
6.9. Тестовый пример Т9 — Отклонение предложения (Ф9) 13
6.10. Тестовый пример Т10 — Обработка ошибок сети (Н3) 13
6.11. Тестовый пример Т11 — Контроль состава экранов (И1) 13
6.12. Тестовый пример Т12 — Локализация уведомлений (И2) 13
ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ 14

1. ОБЪЕКТ ИСПЫТАНИЙ

Объектом испытаний является прикладная программа «Кросс-продуктовая он-чейн система лояльности «LoyalX»» (Cross-Product On-Chain Loyalty System «LoyalX»), обозначение `RU.17701729.09.09-01`, реализующая кросс-продуктовую децентрализованную систему лояльности на базе сети TON в виде Telegram Mini App.

Программа состоит из следующих компонентов:

- клиентская часть — Telegram Mini App (TypeScript / React);
- серверная часть — сервис уведомлений (Node.js);
- он-чейн часть — комплекс смарт-контрактов на языке Tact (Factory, BrandJetton, JettonWallet), развёрнутых в сети TON Testnet.

Исходный код программы размещён в репозитории github.com/arsendulaev/LoyalX.

2. ЦЕЛЬ ИСПЫТАНИЙ

Целью проведения испытаний является проверка соответствия фактических характеристик программы требованиям, изложенным в Техническом задании RU.17701729.09.09-01 ТЗ 01-1 [9], в том числе:

- проверка корректности выполнения функциональных сценариев (регистрация бренда, эмиссия Mint, начисление Earn, списание Burn, обмен Swap, согласование курса);
- проверка корректности взаимодействия клиентской части с подключённым TON Connect-кошельком;
- проверка корректности взаимодействия клиентской части с индексатором TonAPI;
- проверка корректности обработки сообщений смарт-контрактами при штатных и нештатных входных данных;
- проверка соблюдения требований к надёжности (обработка ошибок сети и отклонения транзакций);
- проверка соответствия пользовательского интерфейса требованиям к составу экранов и навигации.

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

3.1. Требования, проверяемые в ходе испытаний

В ходе испытаний подлежат проверке следующие группы требований к программе:

Шифр	Наименование требования	Критерий выполнения
Ф1	Подключение кошелька	По нажатию кнопки «Connect Wallet» открывается окно TON Connect; после подтверждения в кошельке адрес отображается в шапке программы.
Ф2	Регистрация бренда	Форма создания бренда принимает реквизиты, формирует сообщение `CreateBrand`; после подтверждения в кошельке бренд отображается в Brand Dashboard.
Ф3	Эмиссия (Mint)	Сообщение `Mint` принимается мастер-контрактом BrandJetton; баланс получателя в TonAPI увеличивается на указанную сумму с учётом decimals.
Ф4	Начисление по QR-коду (Earn)	Сканирование QR-кода вида `ton://transfer/...?text=earn:<nonce>` инициирует транзакцию с `action = 1`; контракт принимает её только при первом использовании nonce.
Ф5	Списание по QR-коду (Burn)	Программа формирует QR-код, содержащий адрес JettonWallet и сумму; после оплаты пользователем баланс списывается.
Ф6	Обмен (Swap)	При активной паре сообщение `TokenTransfer` с `action = 2` приводит к сжиганию исходного баланса и эмиссии целевого по согласованному курсу.
Ф7	Согласование курса (двусторонний протокол)	`ProposeRate` от A ⟶ `IncomingProposal` у B ⟶ `AcceptRate` у B ⟶ `RateAccepted` у A; обмен в обе стороны активируется автоматически.
Ф8	Согласование курса между своими брендами	При совпадении владельцев формируется пакетная транзакция; обе стороны активируются за одну подпись.
Ф9	Отказ от предложения	`RejectProposal` удаляет предложение из `incomingProposals` и отправляет инициатору уведомление `RateRejected`.
Н1	Защита от двойного начисления	Повторная транзакция с уже использованным nonce отклоняется контрактом BrandJetton; баланс не изменяется.
Н2	Атомарность Swap	При недостаточном балансе или неактивной паре операция полностью откатывается, баллы не списываются.

Шифр	Наименование требования	Критерий выполнения
Н3	Обработка ошибок сети	При недоступности TonAPI клиент отображает уведомление и не блокируется; повторный запрос не приводит к двойному списанию.
И1	Состав экранов интерфейса	Программа содержит экраны: список балансов, Mint, Earn, Burn, Swap, Brand Dashboard, экран курсов, экран входящих предложений, история операций.
И2	Локализация всплывающих уведомлений	Уведомления отображаются на русском языке и содержат корректные шаблоны для всех типов операций.

3.2. Требования к среде испытаний

Среда испытаний должна включать:

- виртуальное устройство либо физический смартфон под управлением Android 8.0+ либо iOS 14.0+;
- мессенджер Telegram актуальной версии;
- криптовалютный кошелёк, поддерживающий TON Connect 2.0 (рекомендуется Tonkeeper);
- два независимых аккаунта Telegram с подключёнными разными кошельками (для проверки сценариев согласования курса между разными владельцами);
- активное соединение с сетью TON Testnet и предварительно полученные тестовые токены TON через faucet.

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

На испытания должны быть представлены следующие документы:

Обозначение	**Наименование**	**ГОСТ**
RU.17701729.09.09-01 ТЗ 01-1	Техническое задание	ГОСТ 19.201-78
RU.17701729.09.09-01 81 01-1	Пояснительная записка	ГОСТ 19.404-79
RU.17701729.09.09-01 34 01-1	Руководство оператора	ГОСТ 19.505-79
RU.17701729.09.09-01 51 01-1	Программа и методика испытаний	ГОСТ 19.301-79
RU.17701729.09.09-01 12 01-1	Текст программы	ГОСТ 19.401-78

Состав документации соответствует требованиям ГОСТ 19.101-77 [1].

5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

5.1. Технические средства

Для проведения испытаний используются следующие технические средства:

- персональный компьютер с операционной системой семейства Linux (Ubuntu 22.04 или новее) либо macOS 13+, не менее 8 ГБ оперативной памяти;
- смартфон под управлением Android 8.0+ либо iOS 14.0+ с установленным мессенджером Telegram актуальной версии;
- активное соединение с сетью Интернет с пропускной способностью не менее 1 Мбит/с.

5.2. Программные средства

Для проведения испытаний используются следующие программные средства:

- среда исполнения Node.js версии 20.0 или новее;
- менеджер пакетов npm версии 10.0 или новее;
- мессенджер Telegram (десктопная либо мобильная версия);
- криптовалютный кошелёк Tonkeeper (либо другой кошелёк, поддерживающий TON Connect 2.0);
- веб-браузер на основе Chromium либо Firefox актуальной версии (для отладочного запуска Mini App вне Telegram);
- утилита `curl` либо HTTP-клиент Postman (для проверки HTTP-интерфейса сервиса уведомлений);
- обозреватель TON Testnet `https://testnet.tonviewer.com/` — для контроля состояния транзакций и балансов.

5.3. Порядок проведения испытаний

Испытания проводятся в следующем порядке:

1. **Подготовительный этап.** Развёртывание программы из репозитория согласно инструкциям файла `README.md`; проверка успешного запуска клиентской и серверной частей; получение тестовых токенов TON через `https://t.me/testgiver_ton_bot`.
2. **Функциональные испытания.** Последовательная проверка функциональных требований Ф1 – Ф9 в соответствии с тестовыми примерами раздела 6.
3. **Испытания на надёжность.** Проверка требований Н1 – Н3 путём подачи на программу некорректных входных данных и моделирования отказов сети.
4. **Испытания интерфейса.** Проверка требований И1 – И2 путём ручного обхода всех экранов программы.
5. **Документирование результатов.** По каждому проведённому испытанию заносится запись в протокол испытаний с указанием результата («выполнено» / «не выполнено») и при необходимости — комментариями.

По результатам испытаний оформляется протокол, подписываемый разработчиком и принимающей стороной.

6. МЕТОДЫ ИСПЫТАНИЙ

Применяются методы функционального тестирования «чёрного ящика» с обращением к смарт-контрактам через TonAPI и контролем фактических состояний с помощью обозревателя сети. Каждый тестовый пример включает: исходные данные, последовательность действий, ожидаемый результат и фактический критерий приёмки.

6.1. Тестовый пример T1 — Подключение кошелька (Ф1)

Исходные данные: программа открыта в Telegram, кошелёк Tonkeeper установлен и содержит тестовые токены TON.

Последовательность действий:

1. нажать кнопку «Connect Wallet» в шапке программы;
2. выбрать кошелёк Tonkeeper в открывшемся окне TON Connect;
3. подтвердить подключение в приложении кошелька.

Ожидаемый результат: в шапке программы отображается сокращённая запись адреса (формат `EQ...XXXX`); пункты меню, требующие подключения, становятся активными.

6.2. Тестовый пример T2 — Регистрация бренда (Ф2)

Исходные данные: кошелёк подключён; на счёте не менее 0,1 TON.

Последовательность действий:

1. перейти в Brand Dashboard, нажать «Создать бренд»;
2. заполнить поля: название = «TestBrand», тикер = «TST», описание = «Описание тестового бренда», URL логотипа = действующая ссылка на изображение;
3. нажать «Подтвердить», подписать транзакцию `CreateBrand` в кошельке.

Ожидаемый результат: спустя 10 – 30 секунд бренд появляется в списке Brand Dashboard; адрес мастер-контракта BrandJetton доступен для копирования.

6.3. Тестовый пример T3 — Эмиссия баллов (Ф3)

Исходные данные: зарегистрирован бренд (T2); известен адрес получателя.

Последовательность действий:

1. открыть экран Mint, выбрать бренд, ввести адрес получателя и сумму = 100,00;
2. подписать сообщение `Mint` в кошельке.

Ожидаемый результат: баланс получателя в обозревателе TonAPI увеличивается на 100,00 единиц с учётом decimals; сумма отображается в истории операций со статусом «Выполнено».

6.4. Тестовый пример T4 — Начисление по QR-коду (Ф4, Н1)

Исходные данные: зарегистрирован бренд; пользователь имеет на счёте достаточно TON для комиссии.

Последовательность действий:

1. оператор формирует QR-код Earn с уникальным `nonce`;
2. пользователь сканирует QR-код, подписывает транзакцию;
3. оператор повторно предъявляет тот же QR-код и пытается провести начисление повторно.

Ожидаемый результат: первое начисление успешно увеличивает баланс пользователя; повторная транзакция отклоняется контрактом, баланс не изменяется, в истории операций повторная транзакция отсутствует либо отображена со статусом «Отклонено».

6.5. Тестовый пример T5 — Списание по QR-коду (Ф5)

Исходные данные: пользователь имеет ненулевой баланс брендовых баллов.

Последовательность действий:

1. оператор открывает экран Burn, выбирает бренд и сумму;
2. программа отображает QR-код, содержащий адрес JettonWallet оператора;
3. пользователь сканирует QR-код в своём кошельке, подтверждает перевод.

Ожидаемый результат: после подтверждения транзакции баланс пользователя уменьшается на указанную сумму; в истории оператора появляется запись Burn.

6.6. Тестовый пример T6 — Согласование курса между разными владельцами (Ф7)

Исходные данные: зарегистрированы два бренда A и B, принадлежащие разным владельцам W_A и W_B.

Последовательность действий:

1. владелец W_A на экране курсов своего бренда вызывает «Предложить курс», указывает бренд B и значение курса = 2000 (соответствует 2 баллам B за 1 балл A);
2. подписывает сообщение `ProposeRate`;
3. владелец W_B видит предложение в разделе «Входящие» и нажимает «Принять», подписывает `AcceptRate`.

Ожидаемый результат: после подтверждения второй транзакции:

- геттер `swapActive(B)` контракта A возвращает `true`;
- геттер `swapActive(A)` контракта B возвращает `true`;
- геттер `proposedRate(A)` контракта B возвращает значение `1 000 000 / 2000 = 500`;
- пара становится доступна для обмена в обе стороны.

6.7. Тестовый пример T7 — Согласование курса между своими брендами (Ф8)

Исходные данные: у одного владельца зарегистрированы два бренда C и D.

Последовательность действий:

1. открыть экран курсов бренда C;
2. выбрать пункт «Установить курс с D» и значение курса;
3. подтвердить пакетную транзакцию (4 сообщения) одной подписью.

Ожидаемый результат: после подтверждения транзакции пара C↔D активна в обе стороны; повторных подтверждений со стороны контрагента не требуется.

6.8. Тестовый пример T8 — Обмен баллов (Ф6, Н2)

Исходные данные: активная пара брендов A↔B; пользователь имеет ненулевой баланс баллов A.

Последовательность действий:

1. открыть экран Swap, выбрать «из A в B», указать сумму = 10,00;
2. подписать `TokenTransfer` с `action = 2`.

Ожидаемый результат: баланс A пользователя уменьшается на 10,00; баланс B увеличивается на величину, рассчитанную по согласованному курсу; операция отображается в истории как «Swap» со ссылкой на хеш транзакции.

Дополнительная проверка (Н2): при попытке обменять сумму, превышающую баланс, либо при отключённой паре транзакция отклоняется контрактом, баланс не изменяется.

6.9. Тестовый пример Т9 — Отклонение предложения (Ф9)

Исходные данные: у бренда B существует входящее предложение от бренда A.

Последовательность действий:

1. открыть раздел «Входящие» в экране курсов бренда B;
2. нажать «Отклонить» против предложения от бренда A;
3. подписать сообщение `RejectProposal`.

Ожидаемый результат: предложение исчезает из списка «Входящие» у бренда B; у бренда A в истории операций появляется запись об отклонении предложения.

6.10. Тестовый пример Т10 — Обработка ошибок сети (Н3)

Исходные данные: программа открыта; кошелёк подключён.

Последовательность действий:

1. отключить интернет-соединение на устройстве;
2. нажать «Обновить балансы»;
3. восстановить соединение и повторить нажатие.

Ожидаемый результат: при отсутствии сети программа отображает всплывающее уведомление об ошибке загрузки и не блокируется; после восстановления соединения данные загружаются корректно; повторная транзакция не формируется.

6.11. Тестовый пример Т11 — Контроль состава экранов (И1)

Исходные данные: программа открыта, кошелёк подключён.

Последовательность действий: последовательный обход всех пунктов нижнего меню и кнопок навигации; фиксация наличия каждого экрана.

Ожидаемый результат: программа содержит экраны: список балансов брендов, Mint, Earn, Burn, Swap, Brand Dashboard, экран курсов, экран входящих предложений, история операций. Все экраны открываются без ошибок и содержат соответствующие интерактивные элементы.

6.12. Тестовый пример Т12 — Локализация уведомлений (И2)

Исходные данные: программа открыта.

Последовательность действий: поочерёдное выполнение операций Mint, Earn, Burn, Swap, ProposeRate, AcceptRate с фиксацией текста всплывающих уведомлений.

Ожидаемый результат: для каждой операции отображается русскоязычное уведомление, корректно описывающее результат (успех / ожидание / ошибка) и содержащее наименование бренда и сумму операции.

ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
9. RU.17701729.09.09-01 ТЗ 01-1. Кросс-продуктовая он-чейн система лояльности «LoyalX». Техническое задание.
10. RU.17701729.09.09-01 81 01-1. Кросс-продуктовая он-чейн система лояльности «LoyalX». Пояснительная записка.
11. RU.17701729.09.09-01 34 01-1. Кросс-продуктовая он-чейн система лояльности «LoyalX». Руководство оператора.
12. Документация TON Blockchain. URL: https://docs.ton.org/ (дата обращения: 07.05.2026).
13. TEP-74: Jettons Standard. URL: https://github.com/ton-blockchain/TEPs/blob/master/text/0074-jettons-standard.md (дата обращения: 07.05.2026).
14. Исходный код программы. URL: https://github.com/arsendulaev/LoyalX (дата обращения: 07.05.2026).

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц в докум.)	№ документа	Входящий № сопроводительного докум. и дата	Подп.	Дата
Изм.	Измененных	Замененных	Новых	Аннулированных					